LeetCode Practice

2 Easy: DFS

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

2
EASY

Patterns (in order)

1. DFS (2)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

2 questions | tap a title to jump to it

Easy questions (2) Go >

DFS (2) Go >

E 1. Island Perimeter LC 463

E 2. Flood Fill LC 733

Easy questions (2)

EASY | TOPIC 1 OF 1

DFS

2 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Island Perimeter LC 463
2. Flood Fill LC 733

EASY | DFS | LeetCode #463 | 1 of 2 | 1/2

Island Perimeter

QUESTION

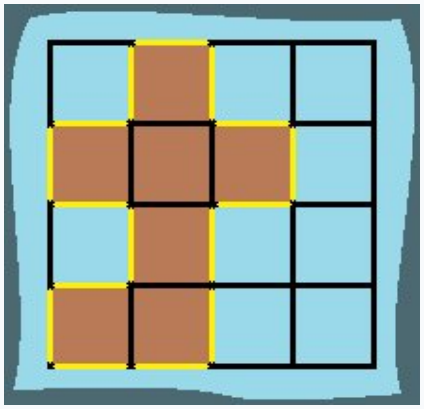
You are given row x col grid representing a map where grid[i][j] = 1 represents land and grid[i][j] = 0 represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1

Input	`grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`
Output	`16`
Explanation	*The perimeter is the 16 yellow stripes in the image above.*



Example 2

Input	`grid = [[1]]`
Output	`4`

Example 3

Input	`grid = [[1,0]]`
Output	`4`

CONSTRAINTS

- row == grid.length
- col == grid[i].length
- 1 <= row, col <= 100
- grid[i][j] is 0 or 1.
- There is exactly one island in grid.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

The grid has exactly one island — no lakes. Each land cell is a unit square.

We return the total perimeter, which is the count of exposed edges. Right?

Exposed means the edge faces water or the grid boundary."

"Quick check: [[1]] -> 4 sides, all exposed -> 4 ok. [[1,0]] -> 4 sides, all exposed -> 4 ok.

Two adjacent land cells: each has 4 sides but share 1 edge -> 2*4 - 2 = 6."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    add 4, minus 1 for each neighbouring land cell
```

TIME	SPACE
O(m * n)	O(1)

NOTE

Not slow: every cell is looked at once, which is optimal. The same answer is 4 * (land cells) - 2 * (adjacent land pairs counted once).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

I notice we're checking 4 neighbours per cell. Instead of neighbour checks,

we can use a counting formula:

Every land cell contributes 4 sides. Every shared edge between two adjacent

land cells removes 2 sides (one from each). So:

`perimeter = islands * 4 - shared_edges * 2`

We only count each shared edge once by checking right-neighbour and down-neighbour.

Same O(m*n) time, same O(1) space — but a cleaner mental model.

PHASE 4 CLEAN CODE

```python
class _463_IslandPerimeter:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        islands = 0
        shared  = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    islands += 1
```

```
                if r + 1 < rows and grid[r + 1][c] ==
1:
                    shared += 1
                if c + 1 < cols and grid[r][c + 1] ==
1:
                    shared += 1
        return islands * 4 - shared * 2
```

PHASE 5 TEST & DEBUG

"Let me trace the perimeter rule step-by-step on the example grids."

Key idea: perimeter = 4*(#land cells) - 2*(#shared edges between adjacent land pairs).

Each shared edge is counted once (right + down), and it removes 1 exposed edge from each of the two cells (so -2 total).

Case 1: grid=[[1]]

Land cells = { (0,0) } -> #land=1 -> 4*(#land)=4.

Shared edges (right+down) = [] -> #shared=0.

Perimeter = 4 - 0 = 4 ok

Case 2: grid=[[1,1]]

Land cells = { (0,0),(0,1) } -> #land=2 -> 4*(#land)=8.

Shared edges:

Right pairs: [((0,0)-(0,1))] -> #shared_right=1

Down pairs: [] -> #shared_down=0

#shared=1 -> Perimeter = 8 - 2*1 = 6 ok

Case 3: grid=[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]

Land cells = { [(0,1),(1,0),(1,1),(1,2),(2,1),(3,0),(3,1)] } -> #land=7 -> 4*(#land)=28.

Shared edges (right+down only):

Right pairs: [(1,0)-(1,1),(1,1)-(1,2),(3,0)-(3,1)] -> #shared_right=3

Down pairs: [(0,1)-(1,1),(1,1)-(2,1),(2,1)-(3,1)] -> #shared_down=3

#shared=6 -> Perimeter = 28 - 2*6 = 16 ok

WHY formula works: every land cell contributes 4 edges to the perimeter; whenever two land cells touch along an edge, that edge stops being exposed for both cells, so we subtract 2 per shared edge.

DFS/count: visited={(0,0),(0,1),(1,0)} (all island cells). Each cell: add 4 - 2*(land_neighbors). component: single island, no need for stack. visited ensures each cell counted once.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	O(1)

The counting formula is elegant — worth presenting even if the brute force also works.

Both approaches are the same complexity; the formula shows mathematical insight.

Given more time I'd ask: what if there are multiple islands?

PHASE 6 COMPLEXITY & FOLLOW-UP

The formula still works per-island if we isolate them, or we can return a list

of perimeters — same logic applied per connected component.

EASY | DFS | LeetCode #733 | 2 of 2 | 2/2

Flood Fill

QUESTION

You are given an image represented by an m x n grid of integers image, where image[i][j] represents the pixel value of the image. You are also given three integers sr, sc, and color. Your task is to perform a flood fill on the image starting from the pixel image[sr][sc].

To perform a flood fill:

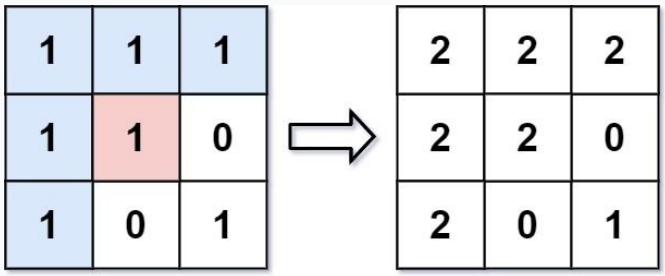
CONSTRAINTS

- 1. Begin with the starting pixel and change its color to color.
- 2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.
- 3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
- 4. The process stops when there are no more adjacent pixels of the original color to update.

Return the modified image after performing the flood fill.

Example 1

Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Explanation	*From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color. Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.*



Example 2

Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0`
Output	`[[0,0,0],[0,0,0]]`
Explanation	*The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.*

- m == image.length
- n == image[i].length
- 1 <= m, n <= 50
- 0 <= image[i][j], color < 2^{16}
- 0 <= sr < m
- 0 <= sc < n

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Starting from (sr, sc), change all connected cells of the same colour to the new colour.

Connected means 4-directionally (up/down/left/right). Modify and return the image. Right?

Edge case: if the starting colour already equals the new colour — return immediately,

otherwise we'd loop infinitely or process unchanged cells."

"[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2:

All 1s connected to (1,1) become 2. The bottom-right 1 is isolated -> stays 1. ok

[[0,0,0],[0,0,0]], sr=0,sc=0, color=0: startcolor==color -> no change -> return as-is ok"

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    scan the whole grid
    any cell next to a recoloured cell with the
        original colour gets the new colour
    if nothing changes:
        break
```

TIME	SPACE
O(m * n * k)	O(1)

WHY IT IS SLOW

Rescans the grid every round (k rounds). A DFS or BFS from the start pixel visits each connected pixel once: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS/BFS from source pixel, recolor connected same-color region — O(mn). Walk-through:"

`source_color = image[sr][sc].`

If source_color == color: return image immediately (already done, avoid infinite loop).

`dfs(r, c):`

If out-of-bounds or image[r][c] != source_color: return.

PHASE 3 OPTIMIZE

image[r][c] = color. <- recolor in-place

dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1).

dfs(sr, sc). Return image.

WHY check source_color == color early: if already the target color, DFS would infinitely recurse

(we'd recolor to color, then neighbors are still 'source_color'... wait: if they're the same,

we'd keep visiting already-visited nodes -> early return prevents this).

Time: O(mn). Space: O(mn) recursion."

Time: O(m×n) — visit each cell at most once. Optimal single-pass.

PHASE 4 CLEAN CODE

```
class _733_FloodFill:
    def floodFill(self, image: List[List[int]], sr: int, s
c: int, color: int) -> List[List[int]]:
        start = image[sr][sc]
        if start == color:
            return image
        rows, cols = len(image), len(image[0])

        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return
            if image[r][c] != start:
                return
            image[r][c] = color     # paint before recursi
ng — acts as visited marker
            dfs(r + 1, c)
            dfs(r - 1, c)
            dfs(r, c + 1)
            dfs(r, c - 1)

        dfs(sr, sc)
        return image
```

PHASE 5 TEST & DEBUG

"Let me trace this DFS step-by-step, including the call stack and visited marking."

image=[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2, start=1:

dfs(1,1): image[1][1]=2. recurse 4 dirs.

dfs(2,1): image[2][1]=1==start -> paint 2. recurse:
dfs(3,1)->out of bounds, dfs(1,1)->now 2!=1 stop.

dfs(2,2)->image[2][2]=1 but that's the isolated corner — wait:

(2,1)->image[2][1]=1ok->paint. Then dfs(2,2)->image[2][2]=1ok->paint? No —

is (2,2) reachable from (1,1)? Path: (1,1)->(2,1)->(2,2).

image[2][2]=1 and image[2][1]=1, so yes connected. But expected output keeps it 1?

Re-check: image=[[1,1,1],[1,1,0],[1,0,1]]. (2,2) is adjacent to (2,1)=0 and (1,2)=0.

So (2,2) connects to (2,1)=0 (water) — not reachable from island. ok

All connected 1s become 2. Bottom-right stays 1 (not connected). ok

PHASE 5 TEST & DEBUG

start==color: image=[[0,0,0],[0,0,0]], color=0 -> return immediately ok

DFS flood fill: visited implicitly via color change (old->new). call stack: dfs(r,c)->dfs(r+1,c)->... component: all connected same-color pixels = one component. Mark on visit.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)**
each cell visited at most once	call stack in the worst case (all cells same colour, grid is a long chain)

The 'paint first then recurse' order is the key — it prevents revisiting and

eliminates the need for a separate visited set. Mention this explicitly.

The early return when startcolor == color prevents infinite looping — critical edge case.

Given more time I'd ask: what if connectivity is 8-directional (diagonals too)?

Add four more direction pairs to the dfs calls — same algorithm.